

## ПОЛИМОРФИЗМ. ВИРТУАЛЬНЫЕ ФУНКЦИИ. АБСТРАКТНЫЕ КЛАССЫ

### Часть 2

В задаче 15 мы создали массив четырехугольников = экземпляров типа **Quad**

Далее мы присваиваем указателю на элемент массива «четырехугольник» указатель на объект «параллелограмм», «квадрат».

И далее вызываем **arr[i].ToString**

Для каждого из элементов нашего массива **arr: array [0..9] of Quad** будет вызвана функция **ToString** класса **Quad**. Потому, что элементы массива относятся к этому классу.

Мы создали объекты дочерних классов, и в этих классах функция **ToString** переопределена.

Однако, переопределенные функции здесь не вызвались.

Но, по условию задачи мы бы хотели увидеть список реальных объектов массива = чтобы вот здесь **writeln(arr[i].ToString)** вызывалась **ToString** того класса, к которому данный объект реально относится.

Как же нам решить эту проблему ?

Мы знаем, что можно сделать явное преобразование типа, используя операцию **as**

Но, для этого мы должны заранее знать тип каждого элемента.

Допустим, что нам не лень написать код проверки типа для каждого элемента массива на соответствие каждому известному дочернему типу. Тогда код будет такой:

```
for var i:=0 to 9 do
begin
    if (arr[i] is Square) then
        writeln( (arr[i] as Square).ToString)
    else if (arr[i] is Parallelogram) then
        writeln( (arr[i] as Parallelogram).ToString)
    else if (arr[i] is Quad) then
        writeln( (arr[i] as Quad).ToString);
end
```

Проблема этого кода не только в том, что нам лень писать его, но еще и в том, что каждый раз, когда в системе появится новый дочерний тип (класс), надо будет не забыть в каждом месте программы расширить этот список.

-----

Итого, мы хотим, чтобы код выглядел так

```
for var i:=0 to 9 do
    writeln(arr[i].ToString);
```

и при этом, чтобы для каждого элемента массива вызывалась «его родная», переопределенная функция **ToString**, а не одна и та же родительская функция **ToString** для всех объектов, вне зависимости от того, какого типа реально этот объект. В этом случае поставленная задача будет решена, каждый элемент массива будет описан правильно.

Есть способ прийти к желаемому результату - использовать **виртуальную функцию**.

**Синтаксис:** нужно добавить слово **virtual** в определении функции в базовом классе и слово **override** в определении функций в дочерних классах

```
function ToString: string; virtual;
```

```
function ToString: string; override;
```

-----

## Про ToString

**ToString** уже является виртуальной функцией базового класса **Object**. Поэтому, в нашем примере достаточно добавить слово **override** для дочерних классов; при этом прямоугольники, составляющие наш массив, могут быть экземплярами класса **Object**.

```
var  
  arr: array [0..9] of Object;
```

---

*И еще про одно синтаксическое упрощение: нам не обязательно явно вызывать **ToString***

### Type

```
Quad = class  
  ...  
public  
  function ToString: string : override;  
  begin  
    Result := '$Coordinates ({p1},{p2},{p3},{p4}), Perimetr: {Perimetr} and  
    Area: {Area}';  
  end;  
end;  
var  
  arr: array [0..9] of object;  
begin  
  ...  
  for var i:=0 to 9 do  
    writeln(arr[i]);  
end.
```

Здесь происходит неявное преобразование типа (наших классов) в string при наличии реализованной у класса функции **ToString**.

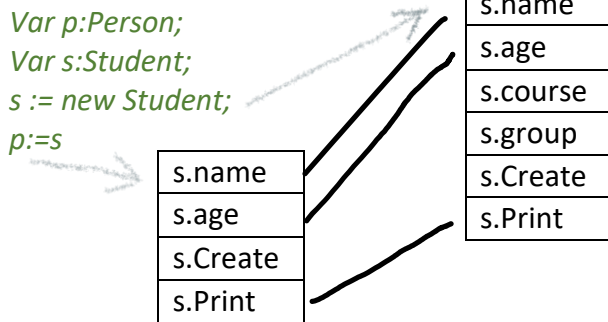
**Упражнение:** Для примера с классами **Person** и **Student** (Тема «Наследование», Часть 1, стр 5-6) опишите функцию Print в базовом классе как виртуальную.

Еще раз смотрим на: **var p: Person := new Student**

Вопрос: чей метод сработает при вызове **p.Print** теперь? Реализованный в **Person** или переопределенный в **Student** ?

**Сначала ответьте, потом проверьте!!!**

Теперь это будет так:



Вопрос: можем ли мы по аналогии с предыдущим кодом (который про прямоугольники) сделать **p** типа **Object** ? Сработает ли такой код?

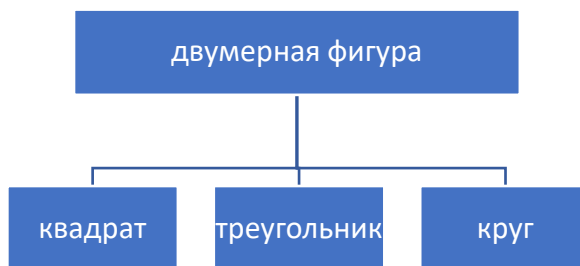
```
var p: Object := new Student;  
p.Print;
```

---

## АБСТРАКТНЫЕ КЛАССЫ.

### *Введение, математическое.*

Порассуждаем на наглядном материале геометрических фигур.  
Пусть у нас есть такая иерархия множеств:



Подумаем про понятие «двумерная фигура». Сформулируйте его.

Задание: представьте объекты (порисуйте их на листочке), составляющие множество двумерных фигур.

Получилось? -

Мы натываемся на то, что нарисовать один **типичный** и при этом конкретный объект «двумерная фигура» мы не можем! Слишком обобщенно (= абстрактно) понятие и описание объектов, составляющих такое множество.

При этом мы можем нарисовать объекты, которые определены не так обобщенно, а более конкретно: круги, прямоугольники, квадраты...

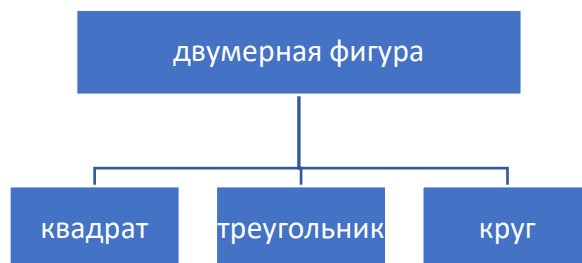
И каждый такой конкретный объект является экземпляром конкретного множества: «круги», «прямоугольники»... И в то же время все конкретные круги и конкретные квадраты являются двумерными фигурами, то есть входят и в множество «двумерные фигуры». Их конкретные свойства – дополнение, конкретизация общих свойств, описанных в понятии «двумерная фигура».

Действительно:

- круг, это И двумерная фигура, И круг
- квадрат, это И двумерная фигура, И квадрат

Далее переходим в термины программирования.

Пусть у нас есть иерархия классов геометрических фигур, соответствующих картинке:



Класс **Shape** «двумерная фигура» - родительский, общий для трех дочерних: квадрат, треугольник, круг. Пусть в каждом из дочерних классов есть процедура рисования объектом самого себя, **procedure Draw**. Название у нее во всех дочерних классах будет такое, а реализация в каждом классе своя собственная.

Мы уже попробовали нарисовать экземпляр класса **Shape** (на листочке) и у нас не получилось. Поэтому нам понятно, что метод **procedure Draw** не может быть реализован в родительском классе **Shape**.

Однако, этот метод есть в дочерних классах, это общее для них всех. А значит, есть логика в том, чтобы метод **Draw** присутствовал в описании родительского класса.

Что будем делать?

Замысел: объявим в базовом классе метод **procedure Draw**, но не будем писать его реализацию, оставим его описание пустым.

Метод **Draw** будет унаследован всеми тремя дочерними классами. В дочерних классах будет прописана конкретная, нужная для данного класса реализация метода **Draw**

Итого, чтобы воплотить замысел:

в базовом классе **Shape**

- мы должны объявить метод **procedure Draw** **виртуальным**

- реализация метода будет пустой, так как мы не знаем, что рисовать

в дочерних классах:

- переопределить **procedure Draw**

- написать его реализацию

**type**

Shape = **class**

**public**

**procedure Draw; virtual;** //объявляем метод виртуальным

**begin**

**end;** // (\*) метод объявлен, реализации нет

**end;**

Circle = **class**(Shape)

**public**

**procedure Draw; override;** //переопределяем виртуальный метод в дочернем классе

**begin**

.....

**end;**

**end;**

Задача 16. Запросите кол-во фигур (кругов и квадратов), которое программа должна вывести на экран. Сгенерируйте случайным образом (как в задаче с четырехугольниками) фигуры и выведите их на экран с использованием GraphABC.

Подсказки:

1) Фигуры храните в массиве. Используйте уже имеющийся у нас шаблонный класс массив:

arr: MyArray<Shape> := new MyArray<Shape>;

2) При рисовании квадратов и кругов используйте процедуры рисования графических примитивов модуля GraphABC:

– DrawRectangle

– DrawCircle

-----  
Смотрим на решение задачи 16.

Пустая реализация (\*) выглядит странно... Есть альтернатива.

Читаем в справочнике:

<https://pascalabc.net/downloads/pabcnethelp/index.htm?page=LangGuide/Classes/abstractmethods.html>

Виртуальные методы (подобные нашему **Draw**):

- которые в родительском классе названы, но не реализованы

- будут реализованы в дочерних классах

называются **абстрактными**.

## СИНТАКСИС

```
type
  Shape = class
    procedure Draw; abstract;
  end;
```

Абстрактные методы - это виртуальные (переопределяемые) методы.

Когда мы используем ключевое слово **abstract** вместо **virtual**, мы избавляемся от необходимости писать виртуальному методу пустую реализацию. Можем теперь просто указать название метода, как и хотелось сделать в примере с классом **Shape**.

Далее, при переопределении абстрактного метода в дочерних классах, мы продолжаем использовать ключевое слово **override**.

Определение: класс, в котором есть хотя бы один абстрактный метод, называется **абстрактным**. Объекты абстрактного класса создавать нельзя.

Привычные нам классы, смысл создания которых – определить специфический тип данных и далее в программе создавать и использовать экземпляры такого типа, называются **конкретными классами**.

---

### Смысл абстрактных классов в программе.

Класс **Shape** – базовый для дочерних.

Он создается не для того, чтобы в программе существовали объекты «двумерная фигура», а с другой целью: он будет **базой иерархии конкретных классов**.

Смысл создания класса **Shape** состоит в том, чтобы **описать в нем все общее, что будет присутствовать во всех дочерних классах**.

Еще раз посмотрим на иерархию классов:



### Вопросы:

- 1) как думаете, может ли быть создан конкретный объект класса **Shape**?
  - 2) можно ли описать в программе переменную абстрактного класса (в разделе **var**)?
  - 3) каким может быть практический смысл существования в программе такого обобщенного класса **Shape**?
-

Резюмируем:

**Конкретные классы:** специфические, конкретно описанные типы данных; объекты такого класса будут создаваться в программе.

**Абстрактные классы:** обобщенные, базовые для дочерних и в этом их смысл. Виртуальные методы названные, но не описанные в абстрактном классе будут реализованы в дочерних конкретных классах.

Переменную абстрактного класса можно описать в программе (создать указатель), но присвоить ей значение можно только через прием *обращение к дочернему объекту как к родительскому* – через присвоение указателя на созданные объекты конкретных дочерних классов.

Верно, также и обратное, и этим можно пользоваться при необходимости: если можно выделить память (вызвать **new**) для переменной, т.е. создать объект класса, значит этот класс не абстрактный

### Задача 17

Создайте класс **Person** и порожденные от него **Student** и **Teacher**.

Запросите список студентов и учителей, сохраните их в общем массиве **MyArray<Person>** и выведите на экран весь список.

Реализуйте для этого **абстрактный** метод **Print**.

Дополнительно посчитайте кол-во студентов и учителей с использованием проверки типа класса **is**.

